

ĐẠI HỌC QUỐC GIA HÀ NỘI - TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

**Hệ thống hiện vật kiểm chứng bằng chứng
không tiết lộ tri thức cho huấn luyện Offline
DQN trên dữ liệu replay đã cam kết**

Học phần: Dự án khoa học

Sinh viên thực hiện:	Nguyễn Ngọc Duy
Mã sinh viên:	23021504
Lớp:	INT3131 70
Giảng viên hướng dẫn:	TS. Nguyễn Ngọc Tân
Ngành:	Khoa học máy tính
Source code:	https://github.com/patee1811/zk_offline_dqn

Hà Nội, tháng 5 năm 2026

Lời cảm ơn

Em xin gửi lời cảm ơn chân thành tới TS. Nguyễn Ngọc Tân, giảng viên hướng dẫn môn Dự án khoa học, đã định hướng đề tài và góp ý trong suốt quá trình thực hiện dự án. Thầy đã giúp em hiểu rõ hơn cách đặt vấn đề nghiên cứu và kiểm soát phạm vi khẳng định sao cho phù hợp với bằng chứng thực tế.

Em cũng cảm ơn các thầy cô Khoa Công nghệ Thông tin, Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội đã trang bị kiến thức nền tảng về học máy, cấu trúc dữ liệu và mật mã, là cơ sở để em tiếp cận đề tài này.

Cuối cùng, em cảm ơn gia đình và bạn bè đã động viên, hỗ trợ em trong quá trình học tập.

Hà Nội, tháng 5 năm 2026
Nguyễn Ngọc Duy

Tóm tắt

Báo cáo trình bày một hệ thống kiểm chứng theo lớp, dùng để xác minh các sản phẩm trung gian trong quá trình huấn luyện Offline Deep Q-Network (DQN) trên tập dữ liệu replay đã được cam kết. Trong học tăng cường ngoại tuyến (Offline RL), bên chứng minh có thể gian lận ở nhiều khâu – từ thành viên replay, phần thưởng, mục tiêu Bellman, cờ kết thúc, lựa chọn hành động, gradient, liên kết checkpoint đến đồng bộ target-network – mà checkpoint cuối cùng vẫn trông hợp lệ. Do đó, hệ thống cần kiểm tra toàn bộ chuỗi tính toán chứ không chỉ kiểm tra kết quả đầu ra.

Hệ thống bao gồm các thành phần: cam kết dữ liệu bằng cây Merkle SHA-256, biểu diễn witness dạng số nguyên (fixed-point) cho phép tính TD và lan truyền thuận/ngịch qua mạng nơ-ron, backend SP1 zkVM để sinh và xác minh bằng chứng không tiết lộ cho các quan hệ đã hỗ trợ, chứng minh đoạn huấn luyện với $k \in \{1, 4, 8\}$ bước, và tổng hợp chuỗi manifest với $T \in \{32, 64, 128\}$ bước. Bên cạnh đó, báo cáo phát biểu tám định lý bảo mật và đối chiếu từng định lý với mã nguồn, backend, test và kết quả thực nghiệm tương ứng.

Thực nghiệm trên các bộ dữ liệu CartPole, MountainCar và PointMaze cho thấy thời gian xác minh luôn dưới 0,21 giây, kích thước bằng chứng dưới 2,84 MB. Bộ kiểm thử chống gian lận gồm 166 trường hợp thuộc 19 nhóm tấn công, không có trường hợp nào bị chấp nhận sai. Hệ thống được thiết kế với phạm vi rõ ràng: chưa bao phủ toàn bộ quá trình huấn luyện DQN, chưa chứng minh bộ tối ưu Adam và chưa hỗ trợ tổng hợp đệ quy thực sự.

Từ khóa: bằng chứng không tiết lộ, học tăng cường ngoại tuyến, học máy có thể kiểm chứng, chứng minh quá trình huấn luyện, Deep Q-Network, SP1 zkVM, cam kết Merkle, phát hiện gian lận.

Lời nói đầu

Trong quá trình tìm hiểu về học tăng cường, em nhận thấy một vấn đề ít được đề cập: làm thế nào để xác minh rằng một mô hình RL thực sự được huấn luyện đúng như mô tả, đặc biệt khi mô hình học từ bộ dữ liệu cố định mà không tương tác thêm với môi trường? Câu hỏi này dẫn em đến ý tưởng áp dụng bằng chứng không tiết lộ tri thức (Zero-Knowledge Proof) vào kiểm chứng huấn luyện Offline DQN.

Báo cáo ghi lại toàn bộ quá trình thực hiện dự án, từ nghiên cứu lý thuyết, thiết kế hệ thống đến triển khai và đánh giá. Mỗi khẳng định kỹ thuật trong báo cáo đều được gắn với mã nguồn, test hoặc kết quả benchmark cụ thể. Em cố gắng phân biệt rõ phần nào đã có bằng chứng mật mã (proof) thực sự và phần nào mới chỉ dừng ở mức kiểm tra logic.

Hệ thống hiện tại còn nhiều giới hạn: chưa chứng minh được toàn bộ quá trình huấn luyện DQN, chưa hỗ trợ bộ tối ưu Adam, chưa có tổng hợp đệ quy thực sự. Những giới hạn này được nêu rõ ở từng phần để tránh hiểu nhầm về khả năng thực tế của hệ thống.

Em rất mong nhận được góp ý từ thầy để hoàn thiện thêm.

Mục lục

Lời cảm ơn	i
Tóm tắt	ii
Lời nói đầu	iii
Danh mục bảng	vi
Danh mục hình	vii
Danh mục từ viết tắt	viii
1 Mở đầu	1
1.1 Lý do chọn đề tài	1
1.2 Mục tiêu của đề tài	1
1.3 Phạm vi và giới hạn	2
1.4 Cấu trúc báo cáo	3
2 Cơ sở lý thuyết và mô hình bài toán	4
2.1 Offline Reinforcement Learning và DQN	4
2.2 Zero-Knowledge Proof trong bài toán kiểm chứng huấn luyện	5
2.3 Merkle Commitment cho dữ liệu replay	6
2.4 Fixed-point arithmetic trong relation layer	6
2.5 SP1 zkVM và cách tổ chức backend	7
2.6 Mô hình bài toán và mô hình đe dọa	8
2.7 Ranh giới riêng tư và ranh giới đúng đắn	9
3 Thiết kế hệ thống và các quan hệ kiểm chứng	10
3.1 Kiến trúc tổng quan	10
3.2 Pipeline dữ liệu và cam kết	10
3.3 Các lớp kiểm chứng trong hệ thống	11
3.4 Các quan hệ kiểm chứng chính	12
3.5 Thiết kế training fragment proof	14
3.6 Proof-manifest chunk-chain aggregation	15

3.7	Theorem và artifact mapping	16
3.8	Tóm tắt thiết kế	18
4	Triển khai hệ thống và thực nghiệm	19
4.1	Tổ chức mã nguồn	19
4.2	Môi trường triển khai	20
4.3	Quy trình chạy và kiểm tra hệ thống	21
4.4	Benchmark RL	22
4.5	Benchmark chi phí ZK proof	23
4.6	Benchmark tamper rejection	24
4.7	Artifact reproducibility	26
4.8	Tổng kết chương	26
5	Đánh giá hệ thống, hạn chế và kết luận	27
5.1	Đánh giá tổng quan	27
5.2	Công trình liên quan	28
5.3	Phân tích ưu điểm của hệ thống	29
5.4	Hạn chế của hệ thống	30
5.5	Bài học rút ra	31
5.6	Hướng phát triển	32
5.7	Kết luận	32
	Tài liệu tham khảo	33
	A Các thuật toán kiểm chứng	36
	Phụ lục A: Các thuật toán kiểm chứng	36

Danh sách bảng

2.1	Phân tách public input và private witness trong hệ thống	5
2.2	Vai trò của các thành phần trong một workspace SP1	7
2.3	Ma trận khẳng định bảo mật: nguy cơ, biện pháp phòng chống, định lý và giới hạn.	8
3.1	Các trường chính trong dataset commitment	11
3.2	Các quan hệ kiểm chứng chính trong hệ thống	13
3.3	Tóm tắt theorem và artifact tương ứng trong hệ thống	16
4.1	Tổ chức mã nguồn chính của dự án	20
4.2	Các công nghệ chính trong quá trình triển khai	21
4.3	Các nhóm dữ liệu trong benchmark RL	22
4.4	Một số kết quả RL performance tiêu biểu	23
4.5	Các metric trong benchmark proof	23
4.6	Một số kết quả proof cost tiêu biểu	24
4.7	Kết quả kiểm thử chống gian lận – đầy đủ 19 nhóm	25
5.1	Tổng hợp các lớp kết quả chính của hệ thống	28
5.2	So sánh định tính với các hệ thống kiểm chứng huấn luyện liên quan	29
5.3	Giới hạn hiện tại và hướng khắc phục	31

Danh sách hình vẽ

2.1	Merkle tree dùng để cam kết các transition trong replay dataset	6
2.2	Ranh giới giữa public input, private witness và relation check	9
3.1	Kiến trúc pipeline kiểm chứng từ dữ liệu replay đến proof và báo cáo thực nghiệm	10
3.2	Các lớp kiểm chứng trong hệ thống	12
3.3	Training fragment proof với chuỗi checkpoint và kiểm tra target-network sync	15
3.4	Proof-manifest chunk-chain aggregation cho trace dài hơn	15
4.1	Đường chạy nhanh của artifact reproducibility	22
4.2	So sánh prove time giữa các relation proof-backed	24
5.1	Các hướng phát triển từ artifact hiện tại tới proof cho trace dài hơn	32

Danh mục từ viết tắt

Viết tắt	Nghĩa đầy đủ
RL	Reinforcement Learning – Học tăng cường
DQN	Deep Q-Network – Mạng Q sâu
MDP	Markov Decision Process – Quá trình quyết định Markov
TD	Temporal-Difference – Sai khác thời gian
SGD	Stochastic Gradient Descent – Hạ gradient ngẫu nhiên
MLP	Multi-Layer Perceptron – Perceptron đa lớp
ZKP	Zero-Knowledge Proof – Bằng chứng không tiết lộ tri thức
zkVM	Zero-Knowledge Virtual Machine – Máy ảo không tiết lộ
STARK	Scalable Transparent Argument of Knowledge
SNARK	Succinct Non-interactive Argument of Knowledge
CQL	Conservative Q-Learning – Học Q bảo toàn
IQL	Implicit Q-Learning – Học Q ẩn
BC	Behavioral Cloning – Nhân bản hành vi
D4RL	Datasets for Deep Data-Driven Reinforcement Learning

Chương 1

Mở đầu

1.1 Lý do chọn đề tài

Khi triển khai mô hình học tăng cường vào các lĩnh vực nhạy cảm như lái xe tự động hay điều khiển robot, một câu hỏi tự nhiên được đặt ra: làm sao biết mô hình thật sự được huấn luyện đúng? Kết quả đầu ra tốt trên vài phép thử chưa đủ, bởi dữ liệu huấn luyện, phép tính trung gian hay chuỗi cập nhật trọng số đều có thể đã bị sửa đổi mà ta không hay biết.

Vấn đề này đặc biệt nghiêm trọng trong Offline RL. Ở đây, mô hình học hoàn toàn từ một tập dữ liệu replay cố định, không tương tác thêm với môi trường. Dữ liệu replay trở thành nguồn thông tin duy nhất, nên nếu tập replay bị can thiệp thì mọi kết quả phía sau đều bị ảnh hưởng.

Với DQN, mỗi bước cập nhật phụ thuộc vào nhiều thành phần đan xen: lấy mẫu transition, kiểm tra transition có thuộc dataset đã cam kết hay không, tính Bellman target, chọn hành động bằng online network, đánh giá giá trị bằng target network, tính loss, lan truyền gradient rồi cập nhật trọng số. Chỉ cần thay đổi reward, done flag, Q value hay gradient ở một bước, checkpoint cuối cùng vẫn có thể trông hợp lệ nhưng thực tế đã sai.

Xuất phát từ bài toán đó, em xây dựng một hệ thống kiểm chứng theo lớp cho Offline DQN. Ý tưởng cốt lõi là chia bài toán thành các phần nhỏ – cam kết dữ liệu, quan hệ kiểm chứng, proof backend, đoạn huấn luyện, tổng hợp – rồi gắn mỗi phát biểu với một sản phẩm cụ thể có thể truy vết được.

Về mặt kỹ thuật, dự án kết hợp ba thành phần chính. Merkle commitment ràng buộc tập replay với các thông tin đi kèm như manifest, audit report, trajectory gốc. Fixed-point arithmetic loại bỏ sai khác do số thực dấu phẩy động khi chuyển logic từ Python sang Rust. SP1 zkVM sinh và xác minh bằng chứng mật mã cho các quan hệ đã hỗ trợ.

1.2 Mục tiêu của đề tài

Mục tiêu chính là xây dựng một pipeline kiểm chứng có thể tái lập cho các đoạn huấn luyện Offline DQN. Pipeline này không dừng ở việc kiểm tra kết quả đầu ra, mà theo dõi cả mối quan hệ giữa dữ liệu, phép tính, checkpoint và bằng chứng mật mã.

Cụ thể, dự án đặt ra ba nhóm mục tiêu:

Về dữ liệu: Xây dựng quy trình thu thập hoặc import dataset, kiểm tra dữ liệu (audit), rồi tạo cam kết Merkle. Với dữ liệu tự thu thập, hệ thống kiểm tra replay và reward trước khi cam kết. Với dữ liệu công khai (Minari/D4RL), chỉ kiểm tra tính toàn vẹn nguồn chứ không

coi đó là bằng chứng thu thập trung thực.

Về quan hệ kiểm chứng: Mô hình hóa các bước quan trọng của DQN thành các quan hệ kiểm tra được, gồm: thành viên replay, mục tiêu TD, lan truyền thuận qua MLP, SmoothL1 loss, gradient, cập nhật SGD, liên kết checkpoint và đồng bộ target-network. Các quan hệ trọng yếu được đưa sang backend SP1 khi đã có đủ bằng chứng sinh proof, xác minh proof, đầu vào công khai và kết quả tamper.

Về thực nghiệm: Xây dựng ba nhóm benchmark. Nhóm đầu đánh giá hiệu năng RL trên dataset đã cam kết. Nhóm hai đo chi phí sinh và xác minh proof. Nhóm ba kiểm tra khả năng phát hiện gian lận ở nhiều lớp. Ba nhóm kết quả này giúp người đọc phân biệt rõ phần nào đã có bằng chứng mật mã và phần nào chỉ là kiểm tra logic.

1.3 Phạm vi và giới hạn

Dự án tập trung vào các đoạn huấn luyện được chọn của Offline DQN, không phải toàn bộ quá trình huấn luyện từ khởi tạo đến checkpoint cuối cùng. Một proof đầy đủ sẽ phải kiểm tra lịch lấy mẫu, toàn bộ replay batch, trạng thái optimizer, quá trình chọn mô hình – điều này nằm ngoài phạm vi hiện tại.

Dự án chỉ khẳng định về các quan hệ và cấu hình đã có sản phẩm kiểm chứng đi kèm. Cụ thể: các quan hệ SP1 trên vector chuẩn, cập nhật một bước, đoạn huấn luyện với $k \in \{1, 4, 8\}$ bước, và tổng hợp chuỗi manifest với $T \in \{32, 64, 128\}$ bước.

Các giới hạn cần nêu rõ:

- Chưa chứng minh toàn bộ quá trình huấn luyện DQN từ khởi tạo đến checkpoint cuối.
- Chưa hỗ trợ bộ tối ưu Adam – chỉ có SGD đơn giản trên mạng nhỏ.
- Chưa có bằng chứng mật mã cho đoạn huấn luyện dài ($k \in \{16, 32, 128\}$).
- Chưa hỗ trợ batch-size lớn (chỉ batch-size 1).
- Tổng hợp hiện tại là chuỗi manifest, chưa phải tổng hợp đệ quy thực sự.
- Với dữ liệu công khai Minari/D4RL, chỉ kiểm tra tính toàn vẹn nguồn, không chứng minh thu thập trung thực.

Mỗi phần trong báo cáo đều cần đường dẫn kiểm chứng cụ thể: mã nguồn quan hệ, backend, test, kết quả benchmark hoặc manifest. Cách trình bày này giữ cho khẳng định không vượt quá bằng chứng đã có.

1.4 Cấu trúc báo cáo

Báo cáo gồm năm chương. Chương 1 giới thiệu đề tài, mục tiêu và phạm vi. Chương 2 trình bày cơ sở lý thuyết gồm Offline RL, DQN, Merkle commitment, Zero-Knowledge Proof, SP1 zkVM và mô hình đe dọa. Chương 3 mô tả thiết kế hệ thống, các quan hệ kiểm chứng, đoạn huấn luyện và tổng hợp chuỗi. Chương 4 trình bày triển khai, mã nguồn và ba nhóm benchmark. Chương 5 đánh giá kết quả, nêu hạn chế và hướng phát triển.

Chương 2

Cơ sở lý thuyết và mô hình bài toán

2.1 Offline Reinforcement Learning và DQN

Trong học tăng cường ngoại tuyến (Offline RL), mô hình không tương tác trực tiếp với môi trường mà chỉ học từ một tập dữ liệu thu thập từ trước. Ở thiết lập này, dữ liệu replay là nguồn thông tin duy nhất để mô hình học hành vi. Do đó, nếu tập dữ liệu bị chỉnh sửa hoặc không phản ánh đúng quá trình thu thập ban đầu, toàn bộ kết quả huấn luyện phía sau sẽ bị sai lệch.

Một transition trong replay buffer được biểu diễn dưới dạng:

$$\tau_i = (s_i, a_i, r_i, s'_i, d_i),$$

với s_i là trạng thái, a_i là hành động, r_i là phần thưởng, s'_i là trạng thái kế tiếp và $d_i \in \{0, 1\}$ là cờ kết thúc (done flag). Khi đưa vào hệ thống kiểm chứng, mỗi transition không chỉ là mẫu huấn luyện bình thường mà còn là một đối tượng cần xác minh: ta phải chắc chắn transition đó thực sự thuộc về tập dữ liệu ban đầu và các trường bên trong không bị sửa đổi.

Deep Q-Network học một hàm giá trị hành động $Q_\theta(s, a)$, trong đó θ là trọng số của mạng. Với một transition (s, a, r, s', d) , mục tiêu Bellman trong DQN cơ bản được tính theo:

$$y = r + \gamma(1 - d) \max_{a'} Q_{\bar{\theta}}(s', a'),$$

trong đó γ là hệ số chiết khấu và $Q_{\bar{\theta}}$ là target network. Thành phần $(1 - d)$ có vai trò loại bỏ giá trị tương lai khi transition đi tới trạng thái kết thúc. Nếu d bị sửa từ 1 thành 0, hệ thống có thể cộng thêm giá trị tương lai không hợp lệ vào mục tiêu huấn luyện.

Trong Double DQN, hành động tốt nhất ở trạng thái kế tiếp được chọn bởi online network, còn giá trị của hành động đó được đánh giá bằng target network:

$$a^* = \arg \max_{a'} Q_\theta(s', a'), \quad y = r + \gamma(1 - d) Q_{\bar{\theta}}(s', a^*).$$

Cách tách action selection và target evaluation giúp giảm overestimation, nhưng đồng thời làm quan hệ kiểm chứng phức tạp hơn. Một proof hợp lệ không chỉ cần kiểm tra giá trị Q , mà còn phải kiểm tra rằng hành động a^* thật sự là kết quả argmax của online network và giá trị target được lấy đúng từ target network.

Trong dự án này, loss được mô hình hóa bằng SmoothL1:

$$L(\delta) = \begin{cases} \frac{1}{2}\delta^2, & \text{nếu } |\delta| < 1, \\ |\delta| - \frac{1}{2}, & \text{ngược lại,} \end{cases}$$

với $\delta = Q_\theta(s, a) - y$. Việc dùng SmoothL1 phù hợp với DQN vì hàm này ổn định hơn squared error khi TD error lớn. Trong hệ thống kiểm chứng, nhánh của SmoothL1 cũng phải được kiểm tra rõ ràng, vì chỉ cần chọn sai nhánh thì gradient và bước cập nhật phía sau sẽ sai.

2.2 Zero-Knowledge Proof trong bài toán kiểm chứng huấn luyện

Bằng chứng không tiết lộ tri thức (Zero-Knowledge Proof - ZKP) cho phép một bên chứng minh (prover) thuyết phục bên xác minh (verifier) rằng một phát biểu là đúng mà không cần tiết lộ toàn bộ dữ liệu bí mật (witness). Dưới dạng tổng quát, một quan hệ (relation) được viết là:

$$R(x, w) = 1,$$

trong đó x là public input (đầu vào công khai) và w là private witness. Verifier sẽ chấp nhận nếu prover đưa ra được bằng chứng hợp lệ.

Áp dụng vào bài toán của dự án, public input thường là các giá trị dùng để ràng buộc như dataset root, manifest hash hay checkpoint hash. Trong khi đó, private witness là các chi tiết cần giấu, ví dụ như nội dung transition, đường dẫn Merkle hay trọng số mô hình ở các bước trung gian.

Bảng 2.1: Phân tách public input và private witness trong hệ thống

Public input	Private witness
Dataset root, Merkle root	Transition field values
Manifest hash, audit report hash	Merkle sibling path
Raw trajectory hash, config hash	Model weights và intermediate tensors
Input/output checkpoint hash	Gradient tensors
Relation identifier, proof public values	Fixed-point intermediate values

Cần lưu ý rằng ZKP trong dự án này không có nghĩa là che giấu mọi thứ. Những thông tin đã được đưa vào public input hoặc báo cáo benchmark thì vẫn là dữ liệu công khai. Tính riêng tư chỉ áp dụng cho các trường witness bên dưới. Do đó, việc thiết kế quan hệ kiểm chứng đòi hỏi sự cẩn thận để tách biệt rõ ràng phần nào cần giấu và phần nào cần công bố.

2.3 Merkle Commitment cho dữ liệu replay

Merkle tree là cấu trúc hash tree dùng để cam kết một tập dữ liệu lớn bằng một root ngắn. Trong hệ thống này, mỗi transition được canonical encode, sau đó hash thành một leaf. Các leaf được ghép cặp và hash dần lên tới root. Khi cần chứng minh một transition thuộc dataset, prover chỉ cần cung cấp leaf, index và sibling path; verifier có thể tái tính root mà không cần đọc toàn bộ dataset.

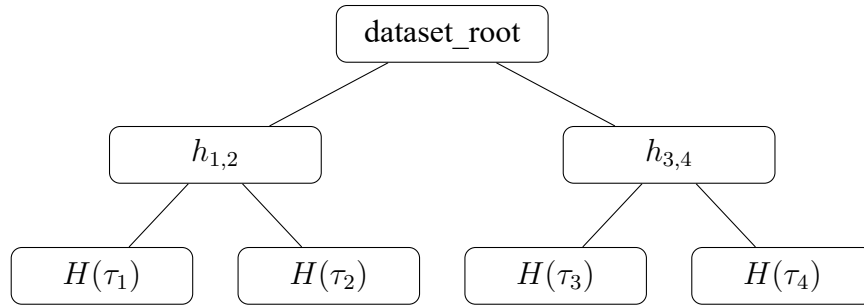
Với một transition τ_i , leaf hash được tính theo:

$$\ell_i = H(\text{canonical_encode}(\tau_i)).$$

Nếu sibling path của ℓ_i là π_i , relation membership kiểm tra:

$$R_{\text{mem}}(\text{root}; \tau_i, \text{idx}_i, \pi_i) = 1 \iff \text{MerkleVerify}(\text{root}, \ell_i, \text{idx}_i, \pi_i) = 1.$$

Như vậy, nếu reward, action, next state, done flag, leaf index hoặc một sibling hash bị thay đổi, root tái tính sẽ không còn khớp với dataset root công khai.



Hình 2.1: Merkle tree dùng để cam kết các transition trong replay dataset

Khác với cây Merkle thông thường, dự án còn gắn kết root của dữ liệu với các thông tin nguồn gốc (provenance). Dataset commitment được biểu diễn như sau:

$$C_D = (h_{\text{root}}, h_{\text{manifest}}, h_{\text{audit}}, h_{\text{raw}}, h_{\text{collection}}).$$

Trường $h_{\text{collection}}$ chỉ có mặt ở tập dữ liệu tự thu thập. Đối với tập dữ liệu công khai, hệ thống chỉ dùng để kiểm tra tính toàn vẹn của nguồn dữ liệu (source-integrity) chứ không coi đó là minh chứng cho việc thu thập trung thực.

2.4 Fixed-point arithmetic trong relation layer

Một thách thức lớn khi cài đặt logic kiểm chứng từ Python sang Rust/SP1 là sai số do dấu phẩy động (floating-point). Trong machine learning thông thường, sai số rất nhỏ có thể bỏ qua. Tuy nhiên, trong hệ thống proof, cùng một input phải cho ra chính xác cùng một kết quả ở cấp độ

bit. Để giải quyết bài toán này, em đã dùng số học dấu phẩy tĩnh (fixed-point arithmetic).

Bằng cách nhân với hệ số scale $S = 1000$, một số thực v được làm tròn thành số nguyên:

$$v_{\text{fp}} = \lfloor v \cdot S \rfloor.$$

Khi nhân hai số fixed-point, ta phải chia lại cho S để đưa về đúng tỷ lệ:

$$(a_{\text{fp}} \cdot b_{\text{fp}})_{\text{rescaled}} = \left\lfloor \frac{a_{\text{fp}} b_{\text{fp}}}{S} \right\rfloor.$$

Nhờ cách biểu diễn này, Python và Rust/SP1 guest luôn sử dụng chung một logic tính toán. Điều này cũng làm rõ phạm vi của hệ thống: dự án không chứng minh quá trình huấn luyện PyTorch nói chung, mà chỉ chứng minh tính đúng đắn của các phép toán fixed-point đã được cài đặt.

Đồng thời, cách làm này giới hạn phạm vi proof ở kiến trúc mạng Linear–ReLU–Linear trên số nguyên, không phải là bất kỳ mạng neural nào. Đây là đánh đổi cần thiết để hệ thống chạy ổn định và có tính xác định.

2.5 SP1 zkVM và cách tổ chức backend

SP1 zkVM là công cụ được chọn để sinh bằng chứng cho các quan hệ viết bằng Rust. Thay vì tự thiết kế các mạch (circuit) phức tạp cho từng phép toán, em chỉ cần viết một chương trình kiểm chứng (guest program) chạy trên zkVM. Sau đó, SP1 sẽ chứng minh rằng chương trình này đã được thực thi đúng.

Mỗi module backend SP1 trong dự án được tổ chức thành ba crate: Shared, Guest và Host. Crate Shared định nghĩa cấu trúc dữ liệu và logic kiểm chứng chung. Crate Guest chạy trong môi trường zkVM. Crate Host chạy bên ngoài để điều phối, nạp dữ liệu (fixture) và ghi nhận các thông số như thời gian chạy, kích thước proof.

Bảng 2.2: Vai trò của các thành phần trong một workspace SP1

Thành phần	Vai trò trong hệ thống
Shared crate	Định nghĩa public input, witness, fixed-point helper và relation logic.
Guest crate	Chạy relation logic trong SP1 zkVM và quyết định accept/reject.
Host crate	Nạp fixture, chạy execute/prove, verify proof và ghi metrics.
Python wrapper	Kết nối backend SP1 với scripts, tests và artifact reports.

Quá trình thực thi có hai chế độ: Execute mode (chỉ kiểm tra logic, không sinh proof) và Prove mode (vừa sinh vừa xác minh proof). Em quy ước một quan hệ chỉ được coi là "proof-

backed”khi nó đã chạy qua Prove mode thành công và có đủ thông số (metrics) đi kèm.

2.6 Mô hình bài toán và mô hình đe dọa

Bài toán đặt ra là: prover cần thuyết phục verifier rằng các bước tính toán TD, cập nhật trọng số và liên kết checkpoint là đúng, mà verifier không cần chạy lại toàn bộ quá trình huấn luyện. Verifier chỉ cần kiểm tra các public input (như dataset root, checkpoint hash) sao cho chúng khớp với một witness hợp lệ. Mục tiêu ở đây không phải là chứng minh mô hình hoạt động tối ưu, mà chỉ là chứng minh các bước tính toán không bị làm giả.

Mô hình đe dọa giả định prover có khả năng và động cơ để gian lận. Họ có thể sửa phần thưởng, đổi Merkle path, can thiệp gradient hay phá vỡ chuỗi checkpoint. Verifier chỉ chấp nhận kết quả nếu các quan hệ kiểm chứng và sự ràng buộc của public input đều hợp lệ.

Bảng 2.3: Ma trận khẳng định bảo mật: nguy cơ, biện pháp phòng chống, định lý và giới hạn.

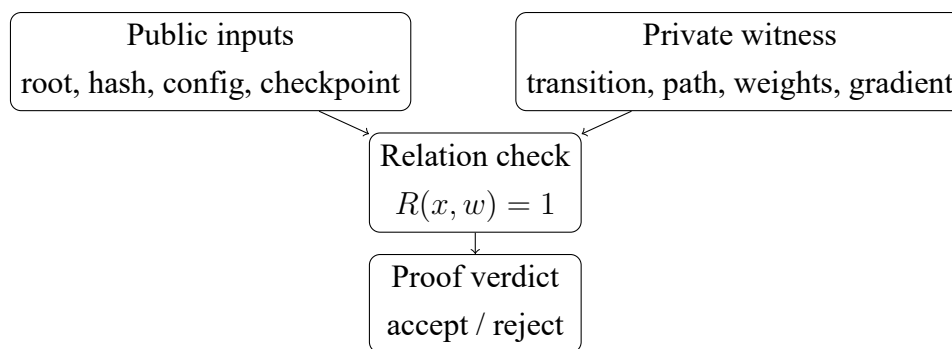
Nguy cơ (Threat)	Biện pháp giảm nhẹ	Định lý / Nhóm kiểm thử	Giới hạn còn lại
Dùng mẫu ngoài tập dữ liệu	Tính lại Merkle path và root	Định lý 1, 2 (Nhóm: merkle)	Không minh chứng thu thập (với data ngoài)
Sửa reward, action, next state, done	Audit dữ liệu và tính lại TD	Định lý 2, 3 (Nhóm: data)	Chỉ audit được reward nếu có log gốc
Làm giả Q value, TD target, loss	Tính lại Bellman và SmoothL1 fixed-point	Định lý 3 (Nhóm: computation)	Không đảm bảo policy đầu ra là tối ưu
Làm giả gradient hoặc SGD update	Kiểm tra backprop và SGD trên fixed-point	Định lý 4 (Nhóm: gradient, ckpt)	Chỉ hỗ trợ batch-size 1 và mạng nhỏ
Phá chuỗi checkpoint, tráo target sync	Liên kết checkpoint và target từng bước	Định lý 5, 6 (Nhóm: trace)	Chưa chứng minh toàn bộ quá trình dài
Đảo lộn các đoạn proof manifest	Ràng buộc mã băm và liên kết checkpoint	Định lý 7 (Nhóm: aggregation)	Chỉ tổng hợp dạng chuỗi manifest
Dùng proof cho public input khác	Ràng buộc public-input chặt chẽ ở host	Định lý 8 (Nhóm: public_input)	Metadata đã công khai không thể giấu

Mô hình này cũng xác định rõ các non-goals. Hệ thống không chứng minh rằng một policy là tối ưu, không chứng minh convergence của DQN, không chứng minh toàn bộ training run từ initialization tới final checkpoint, không chứng minh Adam optimizer và không chứng minh honest collection cho public Minari/D4RL dataset. Những giới hạn này không phải lỗi thiết kế, mà là cách giữ cho claim của hệ thống khớp với artifact đã triển khai.

2.7 Ranh giới riêng tư và ranh giới đúng đắn

Trong hệ thống này, tính riêng tư và tính đúng đắn là hai khái niệm độc lập. Bằng chứng ZKP chỉ giúp che giấu phần private witness, chứ không làm cho các public input trở nên bí mật. Mọi thứ từ dataset root, các mã băm cấu hình, cho đến kết quả benchmark đều là thông tin công khai theo đúng thiết kế.

Về tính đúng đắn của dữ liệu, em cũng chia thành hai ranh giới rõ ràng. Với dữ liệu tự thu thập, hệ thống có thể chạy lại môi trường để kiểm tra phần thưởng và trạng thái trước khi cam kết. Còn với tập dữ liệu công khai, hệ thống chỉ có thể đảm bảo tính toàn vẹn của file tải về (source-integrity). Vì vậy, em không khẳng định dữ liệu công khai đã được thu thập trung thực, mà chỉ coi đó là minh chứng hệ thống có thể chạy trên dữ liệu ngoài.



Hình 2.2: Ranh giới giữa public input, private witness và relation check

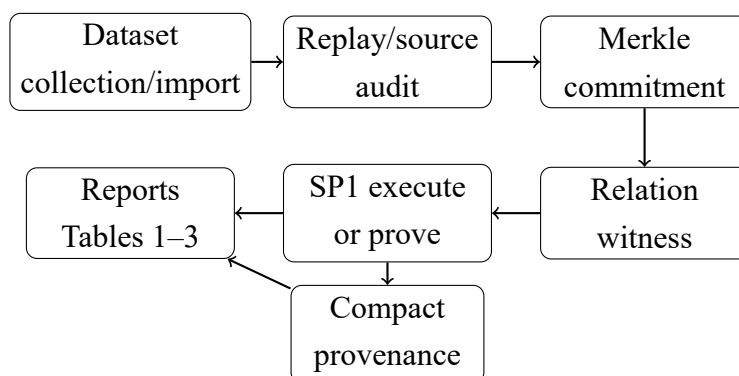
Chương 3

Thiết kế hệ thống và các quan hệ kiểm chứng

3.1 Kiến trúc tổng quan

Hệ thống được thiết kế theo hướng phân lớp, mỗi lớp xử lý một khía cạnh riêng của bài toán. Thay vì dồn mọi thứ vào việc kiểm tra mô hình cuối cùng, em chia nhỏ ra: kiểm tra dữ liệu, kiểm tra các phép toán, sinh proof, và cuối cùng là tổng hợp thành báo cáo. Cách làm này giúp mỗi khẳng định kỹ thuật đều có thể truy vết về một sản phẩm (artifact) cụ thể.

Luồng xử lý bắt đầu từ việc thu thập hoặc tải dữ liệu. Dữ liệu sau đó được kiểm tra rồi đưa vào hệ thống tạo cam kết Merkle. Từ cam kết này, hệ thống sẽ sinh ra witness cho các bước kiểm chứng tiếp theo. Tùy thuộc vào quan hệ (relation), ta có thể kiểm tra nhanh bằng Python, chạy thử ở môi trường Rust (execute mode), hoặc sinh bằng chứng thực sự bằng SP1 zkVM. Kết quả được lưu lại dưới dạng các bảng biểu và báo cáo gọn nhẹ (compact provenance).



Hình 3.1: Kiến trúc pipeline kiểm chứng từ dữ liệu replay đến proof và báo cáo thực nghiệm

Về mặt triển khai, kiến trúc này được phản ánh trực tiếp trong cấu trúc mã nguồn. Các relation ở mức Python nằm trong `zk_offline_dqn/relations`; các verifier bọc quanh fixture và artifact nằm trong `zk_offline_dqn/verifiers`; các helper chạy SP1 ở phía Python nằm trong `zk_offline_dqn/backends/sp1`; còn các chương trình Rust host, guest và shared cho SP1 nằm dưới `zk_backend`. Các script trong `scripts/experiments` chịu trách nhiệm chạy benchmark, sinh bảng kết quả, kiểm tra claim và tạo artifact manifest.

3.2 Pipeline dữ liệu và cam kết

Pipeline dữ liệu được tách thành hai nhánh vì không phải mọi dataset đều có cùng mức bảo đảm. Với self-collected dataset, hệ thống có thể kiểm tra replay và reward trước khi commitment. Với public benchmark dataset như Minari/D4RL, hệ thống chỉ claim source-integrity, tức là

kiểm tra tính nhất quán của nguồn import và artifact đi kèm, không claim rằng quá trình thu thập dữ liệu gốc là trung thực.

Sự phân biệt này quan trọng vì proof sau commitment chỉ chứng minh tính nhất quán với dữ liệu đã được cam kết. Nếu dữ liệu public ban đầu đã sai hoặc không phản ánh quá trình thu thập trung thực, Merkle proof không thể tự biến nó thành honest collection proof. Do đó, hệ thống đặt ranh giới rõ ràng: self-collected dataset có thể mang replay/reward audit evidence, còn public benchmark dataset chỉ được dùng như source-integrity benchmark rows.

Quá trình commitment không chỉ hash các transition. Dataset root được ràng buộc cùng các thông tin provenance như manifest hash, audit report hash, raw trajectory hash và collection log hash khi có. Ở mức khái niệm, commitment có dạng:

$$C_D = (h_{\text{root}}, h_{\text{manifest}}, h_{\text{audit}}, h_{\text{raw}}, h_{\text{collection}}).$$

Trong đó $h_{\text{collection}}$ chỉ xuất hiện khi dataset có collection log tương ứng. Cách ràng buộc này giúp một thay đổi sau commitment ở raw trajectory, manifest, audit report hoặc collection log đều làm sai lệch ít nhất một hash trong artifact.

Bảng 3.1: Các trường chính trong dataset commitment

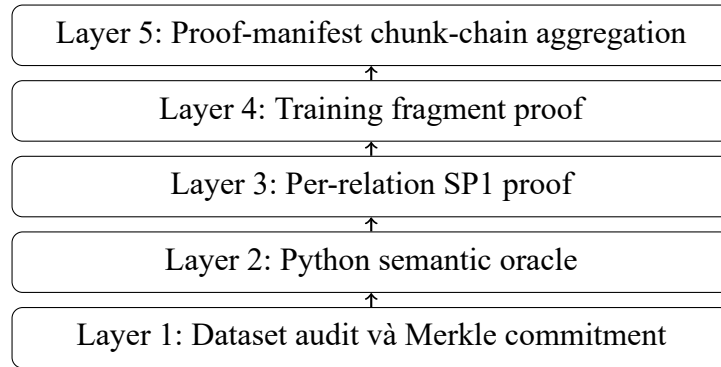
Trường	Vai trò trong hệ thống
Dataset root	Root của Merkle tree được xây từ các transition đã canonical encode.
Manifest hash	Ràng buộc thông tin cấu hình dataset, môi trường, seed và metadata.
Audit report hash	Ràng buộc kết quả kiểm tra replay/reward hoặc source-integrity.
Raw trajectory hash	Ràng buộc dữ liệu trajectory gốc dùng để sinh transition.
Collection log hash	Ràng buộc log thu thập đối với self-collected dataset, nếu có.

Sau khi commitment được tạo, mọi relation liên quan đến dữ liệu phải tham chiếu lại dataset root tương ứng. Điều này làm cho replay membership trở thành điểm nối giữa pipeline dữ liệu và proof backend: một transition chỉ được dùng trong quan hệ TD, training update hoặc training fragment khi nó có thể được mở ra từ root đã cam kết bằng Merkle path hợp lệ.

3.3 Các lớp kiểm chứng trong hệ thống

Hệ thống không chỉ có một tầng bằng chứng duy nhất. Thực tế, em chia các khẳng định thành nhiều lớp với mức chi phí và độ tin cậy khác nhau. Lớp Python (semantic oracle) dùng để kiểm tra nhanh logic ở mức tham chiếu. Lớp Rust (execute mode) chạy thử logic trong môi trường

giống SP1 nhưng chưa sinh proof để tiết kiệm thời gian. Cuối cùng, SP1 prove mode mới là lớp sinh bằng chứng mật mã (cryptographic proof) thực sự.



Hình 3.2: Các lớp kiểm chứng trong hệ thống

Việc chia lớp giúp em giữ các báo cáo kết quả luôn chính xác. Một quan hệ chỉ được gọi là "proof-backed"(đã có bằng chứng) khi nó có đủ khả năng sinh proof, xác minh proof, đi kèm public input, bảng cấu trúc witness và kết quả kiểm thử chống gian lận. Nếu chỉ dừng ở mức Python hay Rust execute thì nó vẫn có giá trị kiểm thử, nhưng sẽ không được tính là khẳng định mật mã trong báo cáo này.

3.4 Các quan hệ kiểm chứng chính

Các quan hệ kiểm chứng (relation) được thiết kế để chặn những kẻ hớ dễ bị gian lận nhất. Nhóm đầu tiên kiểm tra xem dữ liệu có đúng là lấy từ bộ gốc không; nhóm thứ hai kiểm tra tính toán mục tiêu Bellman và sai số TD; nhóm thứ ba kiểm tra quá trình lan truyền ngược và cập nhật trọng số; nhóm cuối cùng lo việc nối các đoạn huấn luyện lại với nhau một cách an toàn.

Bảng 3.2: Các quan hệ kiểm chứng chính trong hệ thống

Quan hệ	Backend hiện tại	Vai trò
Merkle membership	SP1	Kiểm tra transition thuộc dataset đã cam kết bằng Merkle path.
TD MVP	SP1	Kiểm tra mục tiêu Bellman, sai số TD và loss ở dạng tối giản (Q value là witness tự do).
Forward-TD MLP	SP1	Ràng buộc Q value với quá trình tính toán của mạng Linear-ReLU-Linear.
One-step SGD tiny	SP1	Kiểm tra một bước cập nhật SGD nhỏ trên vector chuẩn.
Short trace	SP1	Kiểm tra liên kết checkpoint ngắn để chống đổi checkpoint trung gian.
Training update	SP1	Ghép nối việc kiểm tra dữ liệu, forward TD, tính loss, backpropagation và cập nhật SGD.
Training fragment	SP1 với $k \in \{1, 4, 8\}$	Kiểm tra chuỗi nhiều bước update liên tiếp, việc chọn batch và đồng bộ target network.
Training aggregation	SP1 với $T \in \{32, 64, 128\}$	Ràng buộc chuỗi các manifest đã được verify thông qua mã băm và liên kết checkpoint.

Quan hệ Merkle membership là điểm vào của dữ liệu. Với transition τ_i , index idx_i và path π_i , relation kiểm tra:

$$R_{\text{mem}}(\text{root}; \tau_i, idx_i, \pi_i) = 1 \iff \text{MerkleVerify}(\text{root}, H(\tau_i), idx_i, \pi_i) = 1.$$

Nếu transition không thuộc dataset root, hoặc nếu path bị sửa, relation sẽ không tái tạo được root công khai.

Quan hệ TD kiểm tra mục tiêu Bellman và TD error trên fixed-point arithmetic:

$$y = r_{\text{fp}} + \frac{\gamma_{\text{fp}}(1-d)Q_{\theta, \text{fp}}(s', a^*)}{S},$$

$$\delta = Q_{\theta, \text{fp}}(s, a) - y, \quad \ell = L_{\text{SmoothL1, fp}}(\delta).$$

Ở các relation model-grounded như Forward-TD MLP, training update và training fragment, Q value không còn là witness tự do mà được suy ra từ trọng số mạng đã được cam kết.

Quan hệ Forward-TD MLP kiểm tra forward pass của mạng Linear–ReLU–Linear:

$$Q_\theta(s) = W_2 \cdot \max(W_1 s + b_1, 0) + b_2.$$

Tất cả phép nhân trong biểu thức này đều được rescale theo hệ số fixed-point S . Relation cũng kiểm tra action a^* là kết quả argmax của online network ở trạng thái kế tiếp, sau đó lấy giá trị target tương ứng từ target network.

Quan hệ training update mở rộng Forward-TD MLP bằng backpropagation và SGD update. Với đạo hàm SmoothL1:

$$\frac{\partial L}{\partial \delta} = \begin{cases} \delta, & \text{nếu } |\delta| < S, \\ \text{sign}(\delta) \cdot S, & \text{ngược lại,} \end{cases}$$

hệ thống tính lại gradient qua mạng Linear–ReLU–Linear, tính SGD delta và cập nhật trọng số:

$$W' = W - \eta \frac{\nabla W}{S}.$$

Checkpoint sau cập nhật phải thỏa mãn:

$$H(W') = h_{\text{checkpoint,out}}.$$

Nếu gradient, learning rate, weight delta hoặc checkpoint hash bị thay đổi, relation sẽ reject.

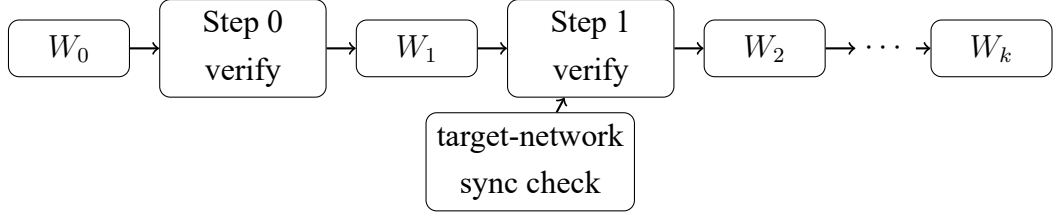
3.5 Thiết kế training fragment proof

Training fragment proof là bước chuyển từ kiểm tra một update đơn lẻ sang kiểm tra một chuỗi update ngắn. Mỗi fragment gồm k bước, với các cấu hình proof-backed hiện tại là $k \in \{1, 4, 8\}$. Trong mỗi bước, hệ thống lấy minibatch index theo deterministic sampler, mở transition tương ứng từ Merkle root, tính lại forward pass, Bellman target, SmoothL1 loss, gradient, SGD update và checkpoint hash.

Một training fragment hợp lệ phải duy trì liên kết:

$$h_{t+1} = H(W_{t+1}), \quad h_{t+1}^{\text{in}} = h_{t+1}^{\text{out previous}},$$

tức checkpoint đầu vào của bước sau phải đúng bằng checkpoint đầu ra của bước trước. Ngoài online checkpoint, hệ thống còn theo dõi target checkpoint để kiểm tra target-network hard sync tại những interval đã cấu hình.



Hình 3.3: Training fragment proof với chuỗi checkpoint và kiểm tra target-network sync

Điểm đáng chú ý là training fragment không chỉ chứng minh rằng checkpoint cuối cùng tồn tại. Nó chứng minh checkpoint cuối cùng được sinh ra từ một chuỗi update mà từng bước đều được kiểm tra theo cùng relation. Điều này làm giảm khả năng một prover bỏ qua bước trung gian, thay checkpoint hoặc sửa target sync mà vẫn trình bày được một trace hợp lệ.

Các fragment dài hơn như $k \in \{16, 32, 128\}$ đã có reference hoặc execute-mode validation trong quá trình phát triển, nhưng không được xem là SP1 proof-backed vì không có proof generation và verification provenance hoàn chỉnh. Vì vậy, trong báo cáo, các độ dài này chỉ nên xuất hiện ở phần giới hạn hoặc future work, không đưa vào claim chính.

3.6 Proof-manifest chunk-chain aggregation

Training fragment proof với $k = 8$ chỉ bao phủ một đoạn ngắn. Để mô tả trace dài hơn, hệ thống dùng proof-manifest chunk-chain aggregation. Ý tưởng là chia trace thành các chunk, mỗi chunk tương ứng với một child manifest đã được verify bên ngoài. Aggregate relation không chạy lại toàn bộ training arithmetic, mà kiểm tra tính nhất quán giữa các manifest và checkpoint link.

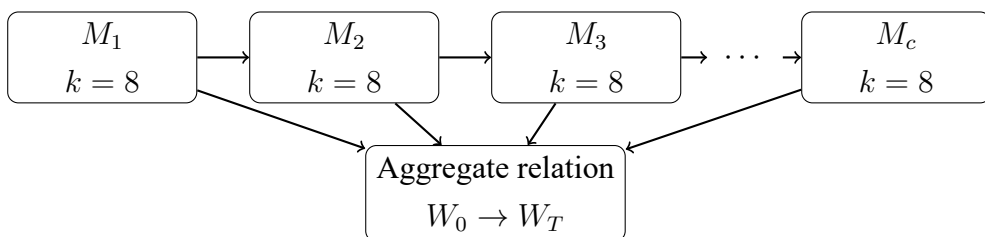
Với c child manifest $M_1, \dots, M_c$, mỗi manifest đại diện cho một fragment $k = 8$. Aggregation kiểm tra các điều kiện chính sau:

$$H(M_j) = h_j,$$

$$M_j.\text{dataset_root} = M_{j+1}.\text{dataset_root}, \quad M_j.\text{config_hash} = M_{j+1}.\text{config_hash},$$

$$M_j.\text{output_checkpoint} = M_{j+1}.\text{input_checkpoint}.$$

Các điều kiện này bảo đảm rằng chuỗi child manifest có cùng dataset/config, đúng thứ tự step range và không bị đứt checkpoint chain.



Hình 3.4: Proof-manifest chunk-chain aggregation cho trace dài hơn

Cần nhấn mạnh rằng đây không phải true recursive aggregation. Aggregate SP1 guest hiện không verify trực tiếp child proof bytes bên trong guest. Child proofs được verify bên ngoài, sau đó đại diện trong aggregate relation bằng proof manifest hash, public-input hash và các checkpoint link. Do đó, claim đúng của hệ thống là proof-manifest chunk-chain aggregation, không phải recursive proof composition.

Aggregation hiện tại ràng buộc child proof manifest, tính nhất quán dataset/config, aggregate roots và checkpoint links. Hệ thống không claim true recursive child-proof verification bên trong SP1 guest.

3.7 Theorem và artifact mapping

Một điểm quan trọng trong thiết kế hệ thống là mỗi theorem phải gắn với artifact cụ thể. Điều này giúp báo cáo tránh tình trạng có phát biểu hình thức nhưng không có mã nguồn, test hoặc benchmark hậu thuẫn. Trong dự án, theorem-artifact mapping được duy trì trong tài liệu riêng và được kiểm tra bằng script, bảo đảm rằng mỗi theorem đều có relation, backend implementation, test và bằng chứng thực nghiệm tương ứng.

Bảng 3.3: Tóm tắt theorem và artifact tương ứng trong hệ thống

Theorem	Nội dung	Artifact tương ứng
Theorem 1	Replay membership soundness	Merkle membership relation, SP1 backend, Merkle path tamper tests, Table 2 và Table 3.
Theorem 2	Audited dataset commitment soundness	Dataset pipeline, commitment verifier, manifest/audit/raw trajectory tamper tests.
Theorem 3	Bellman target correctness	TD MVP, Forward-TD MLP, training update, TD target và Q value tamper tests.
Theorem 4	Forward, backpropagation và update correctness	Training update backend, one-step SGD tiny, gradient và checkpoint tamper tests.
Theorem 5	Checkpoint-chain soundness	Short trace, training fragment, checkpoint hash và step-order tamper tests.
Theorem 6	Training-fragment soundness	Training fragment $k \in \{1, 4, 8\}$, minibatch index và target-network sync tamper tests.

Theorem	Nội dung	Artifact tương ứng
Theorem 7	Proof-manifest chunk-chain aggregation soundness	Training aggregation $T \in \{32, 64, 128\}$, child manifest, public-input hash và checkpoint-link tamper tests.
Theorem 8	Zero-knowledge và privacy boundary	Public/private schema, proof public input binding, witness schema và provenance reports.

Thiết kế này làm rõ ranh giới giữa formal statement và implementation evidence. Một theorem không được hiểu như claim tổng quát cho mọi cấu hình DQN, mà chỉ có hiệu lực trong phạm vi relation, backend và artifact đã triển khai. Vì vậy, khi báo cáo nhắc tới training-fragment soundness, phạm vi đúng là $k \in \{1, 4, 8\}$ với canonical tiny vectors đã có SP1 proof provenance; khi nhắc tới aggregation soundness, phạm vi đúng là proof-manifest chain với $T \in \{32, 64, 128\}$, không phải true recursive aggregation.

Phát biểu theorem chi tiết

Các phát biểu dưới đây tương ứng với phần formal statements của hệ thống. Tất cả theorem đều được hiểu trong phạm vi relation đã triển khai và backend provenance có trong artifact map; chúng không phải phát biểu về một full DQN training run.

Theorem 1: Replay membership soundness. Với một statement Merkle membership được accept, leaf hash của transition đã mở thật sự nằm trong Merkle root đã cam kết theo hash function và path relation đã triển khai. Relation tính lại canonical transition hash, kiểm tra leaf index, path direction và sibling hashes để tái tính root. Nếu transition, sibling, direction, index hoặc root bị sửa, statement sẽ bị reject, trừ trường hợp hash collision hoặc lỗi soundness của proof backend.

Theorem 2: Audited dataset commitment soundness. Nếu dataset commitment verifier accept, thì merkle_tree.json được ràng buộc với manifest hash, audit-report hash, raw trajectory hash, collection-log hash khi có, canonical leaf hashes và dataset root. Verifier hash lại raw episodes, audit report, manifest, collection log đối với self-collected dataset và rebuild toàn bộ Merkle tree. Với public benchmark imports, theorem này chỉ là source-integrity commitment, không phải honest collection proof.

Theorem 3: Bellman target correctness. Với các instance TD MVP, Forward-TD MLP, one-step training update và training fragment được accept, Bellman target, selected Q-value, TD error và loss trong fixed-point phải khớp với relation definition cho transition, action, discount factor, done flag và Q witnesses. Forward-TD MLP và training update còn buộc Q value được suy ra từ trọng số MLP đã cam kết, không phải witness tự do.

Theorem 4: Forward, backpropagation và update correctness. Với một one-step

training-update instance được accept, forward pass Linear–ReLU–Linear, SmoothL1 gradient/backpropagation path và SGD update trong fixed-point phải sinh ra checkpoint hash sau cập nhật đã claim. Theorem này chỉ bao phủ canonical batch-size-1 tiny MLP và one-step SGD tiny vector; nó không bao phủ Adam, batch size 4/8/16 hoặc network tùy ý.

Theorem 5: Checkpoint-chain soundness. Với short-trace và training-fragment instance được accept, các checkpoint liên kế được nối bằng checkpoint hash relation và checked update relation. Verifier tính lại checkpoint hash trước và sau mỗi update, yêu cầu output hash của bước trước khớp input hash của bước sau. Đây là local checkpoint chaining, không phải proof từ initialization tới final deployed model.

Theorem 6: Training-fragment soundness. Với k -step training-fragment proof được accept và $k \in \{1, 4, 8\}$, checkpoint cuối thu được từ chuỗi deterministic sampling, replay membership, Bellman target computation, backpropagation, fixed-point SGD update, checkpoint chaining và target-network hard sync đã triển khai. Các fragment $k \in \{16, 32, 128\}$ chỉ có reference hoặc execute evidence, không phải SP1 proof-backed claim.

Theorem 7: Proof-manifest chunk-chain aggregation soundness. Với aggregation instance được accept và $T \in \{32, 64, 128\}$, aggregate statement ràng buộc một chuỗi child proof manifest $k = 8$ đã được verify bên ngoài, cùng dataset/config hash, chunk boundary, aggregate root và online/target checkpoint link. Đây không phải recursive aggregation theorem: aggregate SP1 guest hiện không chạy verifier cho child proof bytes.

Theorem 8: Zero-knowledge và privacy boundary. Với các relation SP1 proof-backed, verifier chỉ học public inputs và public values mà relation/proof artifact cố ý công bố. Private witness như transition đã mở, Merkle path, weights và tensor trung gian vẫn riêng tư theo interface của proof system, trừ khi chúng được đưa vào public artifact hoặc report. Timing, proof size, dataset root, config hash, checkpoint hash và các metric benchmark là public metadata theo thiết kế.

3.8 Tóm tắt thiết kế

Chương này đã trình bày thiết kế tổng thể của hệ thống kiểm chứng theo lớp cho Offline DQN. Thiết kế không đặt toàn bộ niềm tin vào một proof duy nhất, mà chia bài toán thành các phần có thể kiểm chứng độc lập: dataset commitment, relation semantics, SP1 proof backend, training fragment và proof-manifest aggregation. Cách chia này phù hợp với phạm vi của dự án, vì mỗi claim cụ thể đều có artifact hậu thuẫn và vẫn giữ rõ các giới hạn như chưa chứng minh full DQN training, Adam optimizer và true recursive aggregation.

Chương 4

Triển khai hệ thống và thực nghiệm

4.1 Tổ chức mã nguồn

Mã nguồn của dự án được tổ chức theo hướng tách rõ giữa logic kiểm chứng, backend sinh proof, script thực nghiệm và artifact báo cáo. Mỗi claim trong báo cáo có thể truy ngược về một phần mã nguồn cụ thể. Nếu một relation chỉ tồn tại ở tầng Python semantic oracle, báo cáo không gọi đó là SP1 proof-backed. Nếu một relation đã có Rust host/guest/shared, proof verification, metrics và tamper evidence, relation đó mới được đưa vào nhóm proof-backed.

Ở mức tổng quan, thư mục `zk_offline_dqn` chứa logic Python của hệ thống, bao gồm relation, verifier, wrapper SP1 và report generator. Thư mục `zk_backend` chứa các workspace Rust/SP1 cho từng relation đã được đưa sang backend proof. Các script trong `scripts/data` và `scripts/experiments` đóng vai trò điều phối pipeline dữ liệu, benchmark, kiểm tra claim và sinh bảng kết quả cuối cùng.

Bảng 4.1: Tổ chức mã nguồn chính của dự án

Thư mục/file	Vai trò trong hệ thống
<code>zk_offline_dqn/relation</code>	Chứa các relation Python semantic oracle cho membership, TD, forward MLP, one-step update, training fragment và aggregation.
<code>zk_offline_dqn/verifiers</code>	Chứa các verifier ở tầng Python, dùng để nạp fixture và gọi relation mà không làm thay đổi ngữ nghĩa kiểm chứng.
<code>zk_offline_dqn/backends/sp1</code>	Chứa wrapper Python để gọi backend SP1, xử lý fixture, command, metrics và compact provenance.
<code>zk_backend/*/sp1</code>	Chứa các workspace Rust host, guest và shared cho từng relation SP1 proof-backed.
<code>scripts/data</code>	Chứa script thu thập, audit, import, commit và verify dataset.
<code>scripts/experiments</code>	Chứa script chạy benchmark, sinh report, kiểm tra claim, kiểm tra theorem-artifact map và tạo artifact manifest.
<code>tests/unit</code>	
<code>tests/negative</code>	Chứa unit test, regression test và tamper test cho từng lớp kiểm chứng.
<code>artifacts/reports/final_ndss</code>	Chứa các bảng kết quả cuối cùng phục vụ báo cáo.
<code>artifacts/reports/provenance/sp1</code>	Chứa compact provenance của các proof SP1 đã verify, gồm public input, witness schema, metrics và verify report.

Việc tách benchmark và proof giúp tránh trộn lẫn số liệu thực nghiệm với logic kiểm chứng. Các script benchmark chỉ đọc kết quả hoặc chạy pipeline theo cấu hình đã định nghĩa, trong khi relation và backend giữ vai trò quyết định tính đúng sai của phép kiểm chứng. Khi bảng kết quả được sinh ra, người đọc có thể phân biệt đâu là số liệu thực nghiệm, đâu là relation logic, và đâu là bằng chứng SP1 đã được xác minh.

4.2 Môi trường triển khai

Hệ thống sử dụng Python cho pipeline dữ liệu, relation oracle, benchmark và report generation. Rust được dùng cho các backend SP1 vì guest program cần chạy trong zkVM với logic xác định. Các proof nặng được chạy trên Kaggle CPU trong quá trình phát triển để đảm bảo có môi trường Linux nhất quán. Ở giai đoạn đóng gói cuối, dự án bổ sung Makefile, Dockerfile, `requirements.lock` và artifact manifest để hỗ trợ tái lập kết quả.

Bảng 4.2: Các công nghệ chính trong quá trình triển khai

Công nghệ	Vai trò trong dự án
Python	Điều phối pipeline dữ liệu, semantic oracle, benchmark, report generation và kiểm tra claim.
Rust	Triển khai relation logic cho shared crate, guest crate và host crate của SP1.
SP1 zkVM	Sinh và xác minh proof cho các relation đã được port sang backend SP1.
Kaggle CPU	Môi trường Linux dùng để chạy các proof và benchmark nặng, đồng thời kiểm tra khả năng tái lập.
Dockerfile	Đóng gói môi trường artifact ở mức cơ bản, không nhúng raw dataset hoặc proof binary.
Makefile	Cung cấp các target reproduce để reviewer chạy lại các kiểm tra chính.

Một điểm cần lưu ý là dự án không commit raw dataset, proof binary hoặc receipt lớn. Thay vào đó, dự án commit compact provenance gồm public input, witness schema, metrics, verify report và hash inventory. Cách làm này giữ repository gọn hơn nhưng vẫn cho phép kiểm tra lại claim proof-backed ở mức artifact.

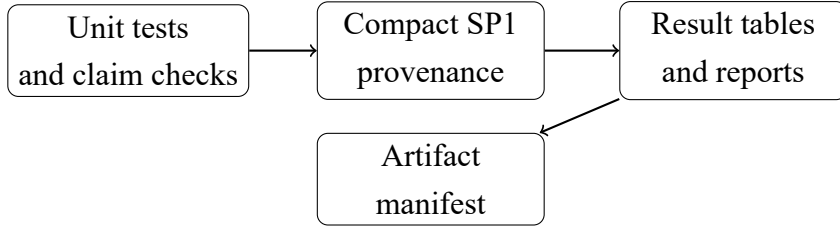
4.3 Quy trình chạy và kiểm tra hệ thống

Quá trình chạy hệ thống được chia thành hai nhóm: kiểm tra nhanh và chạy lại các tác vụ nặng. Kiểm tra nhanh dùng cho quá trình phát triển hằng ngày và artifact review; nhóm tác vụ nặng dùng khi cần sinh lại proof hoặc benchmark từ đầu. Sự phân tách này cần thiết vì SP1 prove có chi phí đáng kể, trong khi kiểm tra claim, kiểm tra report source và validate compact provenance có thể chạy nhanh hơn nhiều.

Đường chạy nhanh được đóng gói bằng lệnh:

```
make reproduce-small
```

Lệnh này không sinh lại toàn bộ proof nặng. Thay vào đó, nó kiểm tra dataset audit smoke path, compact SP1 provenance, theorem-artifact mapping, các báo cáo kết quả và artifact manifest. Đây là đường chạy phù hợp để xác nhận rằng repo không bị thiếu file quan trọng, bảng kết quả không bị lệch nguồn và các claim trong báo cáo vẫn nằm trong phạm vi được hỗ trợ.



Hình 4.1: Đường chạy nhanh của artifact reproducibility

Các đường chạy nặng được bảo vệ bằng biến môi trường như `RUN_HEAVY_SP1`, `RUN_HEAVY_BENCHMARKS` và `RUN_HEAVY_TAMPER`. Thiết kế này tránh việc reviewer vô tình chạy lại toàn bộ proof hoặc benchmark dài, đồng thời vẫn giữ khả năng tái lập khi có đủ môi trường và thời gian.

4.4 Benchmark RL

Benchmark RL được dùng để kiểm tra rằng dataset đã cam kết vẫn hỗ trợ quá trình học ở mức hợp lý. Đây không phải là phần chứng minh cryptographic, nhưng nó cần thiết để báo cáo không chỉ có proof cost mà còn có ngữ cảnh học tăng cường. Nếu dataset không tạo được tín hiệu học nào, việc chứng minh các relation huấn luyện sẽ kém ý nghĩa về mặt ứng dụng.

Dự án sử dụng hai nhóm dữ liệu. Nhóm thứ nhất là self-collected datasets trên CartPole và MountainCar, có replay/reward audit trước khi commitment. Nhóm thứ hai là public PointMaze Minari/D4RL, được xử lý như source-integrity benchmark rows. Sự phân biệt này giữ cho claim dữ liệu chính xác: self-collected datasets có audit evidence, còn public datasets không được xem là honest collection proof.

Bảng 4.3: Các nhóm dữ liệu trong benchmark RL

Nhóm dữ liệu	Vai trò trong thực nghiệm
CartPole self-collected	Dataset có replay/reward audit, dùng để kiểm tra pipeline tự thu thập và khả năng học của các baseline discrete.
MountainCar self-collected	Dataset audited thứ hai, dùng để kiểm tra pipeline trên một môi trường khó hơn với random dataset ít tín hiệu học.
PointMaze Minari/D4RL	Public benchmark dataset, dùng cho các baseline continuous nhưng chỉ claim source-integrity.

Các baseline trong bảng RL gồm Behavior Cloning, Offline DQN, Double DQN, CQL-lite, BC-continuous và IQL-lite. Với CartPole-random, các thuật toán giá trị như Offline DQN, Double DQN và CQL-lite đạt return cao hơn Behavior Cloning, cho thấy dataset có đủ tín hiệu để học chính sách tốt hơn hành vi random ban đầu. Với MountainCar-random, các baseline đều dừng ở mức -200.0 , phản ánh đặc thù của môi trường: random policy hiếm khi tạo ra

trajectory hữu ích để học cách lên đỉnh. Vì vậy, MountainCar trong dự án chủ yếu đóng vai trò artifact kiểm chứng pipeline, không phải bằng chứng về hiệu năng RL mạnh.

Bảng 4.4: Một số kết quả RL performance tiêu biểu

Dataset	Algorithm	Avg Return	Std
carpole-random	Behavior Cloning	11.0	0.9
carpole-random	Offline DQN	161.9	49.9
carpole-random	Double DQN	192.9	69.3
carpole-random	CQL-lite	199.0	34.8
mountaincar-random	Offline DQN	-200.0	0.0
umaze-dense-10k	BC-continuous	186.9	8.1
umaze-dense-10k	IQL-lite	173.8	9.1

Kết quả RL cần được đọc đúng phạm vi. Mục tiêu của benchmark này không phải chứng minh thuật toán RL mới tốt hơn state-of-the-art, mà là kiểm tra rằng các dataset đã đi qua pipeline commitment vẫn có thể dùng để chạy các baseline hợp lệ. Đóng góp của dự án nằm ở khả năng kiểm chứng dữ liệu và computation artifact, không nằm ở việc đề xuất một thuật toán offline RL mới.

4.5 Benchmark chi phí ZK proof

Benchmark proof cost đo chi phí của các relation đã có SP1 proof-backed evidence. Các metric chính gồm prove time, verify time, proof size và cycle count. Trong phạm vi kết quả hiện tại, tất cả cấu hình proof-verified đều có verify time dưới 0.21 giây và proof size dưới 2.84 MB. Điều này cho thấy chi phí verifier khá nhỏ, trong khi chi phí prover vẫn là phần nặng nhất của hệ thống.

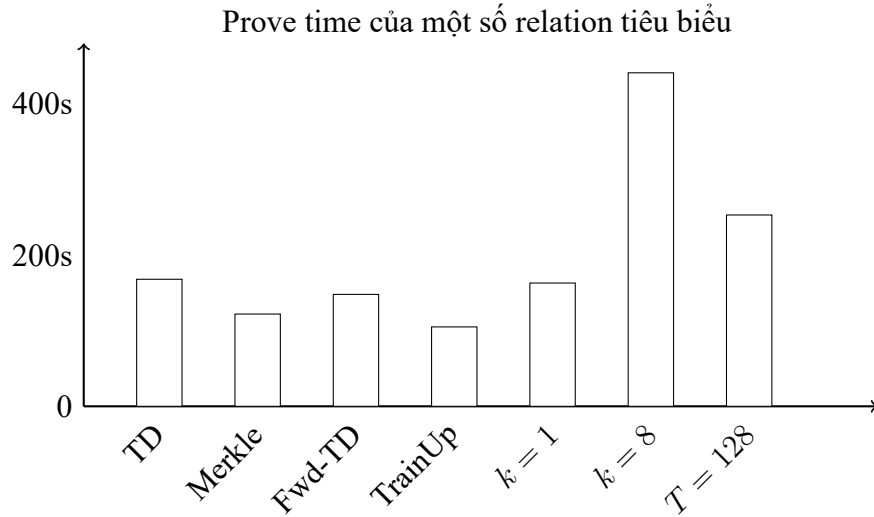
Bảng 4.5: Các metric trong benchmark proof

Metric	Ý nghĩa
Prove time	Thời gian sinh proof cho một relation hoặc một cấu hình cụ thể.
Verify time	Thời gian verifier kiểm tra proof đã sinh.
Proof size	Kích thước proof được tạo ra, thường được báo cáo theo KB hoặc MB.
Cycle count	Số chu kỳ thực thi trong SP1 zkVM, phản ánh độ phức tạp của guest program.
Peak RSS	Bộ nhớ cực đại trong quá trình benchmark, nếu môi trường chạy ghi nhận được.

Bảng proof cost cho thấy các relation đơn lẻ như TD MVP, Merkle membership, Forward-TD MLP, one-step SGD tiny và training update có chi phí khác nhau tùy theo lượng hashing, forward/backward arithmetic và schema logic. Training fragment có xu hướng tăng prove time theo k , vì mỗi fragment lặp lại nhiều bước update. Ngược lại, aggregation tăng chậm hơn theo T , vì aggregate relation chủ yếu kiểm tra hash, manifest và checkpoint link thay vì chạy lại toàn bộ training arithmetic.

Bảng 4.6: Một số kết quả proof cost tiêu biểu

Relation	Prove time	Verify time	Proof size	Cycles
TD MVP	167.7s	0.190s	2,719 KB	385k
Merkle 100k leaves	168.7s	0.195s	2,719 KB	471k
Forward-TD MLP (CartPole)	148.4s	0.127s	2,732 KB	1,547k
Forward-TD MLP (MountainCar)	107.9s	0.127s	2,732 KB	1,547k
Training update	104.8s	0.125s	2,720 KB	469k
Fragment $k = 1$	163.1s	0.155s	2,726 KB	896k
Fragment $k = 8$	440.6s	0.209s	2,771 KB	4,840k
Aggregation $T = 128$	253.2s	0.154s	2,746 KB	2,466k



Hình 4.2: So sánh prove time giữa các relation proof-backed

Kết quả này cho thấy hệ thống hiện phù hợp với prototype nghiên cứu và artifact verification hơn là triển khai production quy mô lớn. Điểm tích cực là verify time thấp và proof size ổn định; điểm hạn chế là prove time tăng rõ rệt với training fragment dài. Vì vậy, các hướng tối ưu trong tương lai cần tập trung vào recursive aggregation thật, GPU proving và cải thiện hiệu quả của relation neural network.

4.6 Benchmark tamper rejection

Benchmark tamper rejection kiểm tra hệ thống trước các thay đổi độc hại có chủ đích. Đây là phần quan trọng vì hệ thống kiểm chứng không chỉ cần chạy được trên input hợp lệ, mà

còn phải reject input bị chỉnh sửa ở đúng lớp. Một tamper có thể bị bắt bởi dataset audit, commitment verifier, Python semantic oracle, Rust execute mode, SP1 verification hoặc public-input binding, tùy theo vị trí tấn công.

Các nhóm tamper bao phủ cả dữ liệu, commitment, computation, trace và verification. Ở tầng dữ liệu, benchmark sửa reward, next state, done flag và action. Ở tầng commitment, benchmark sửa Merkle path, Merkle leaf, dataset root, manifest hash, raw trajectory hash và audit report hash. Ở tầng computation, benchmark sửa Q value, TD target, loss, gradient và checkpoint hash. Ở tầng trace, benchmark sửa minibatch index và target-network sync. Ở tầng verification, benchmark kiểm tra việc dùng proof với public input không khớp.

Bảng 4.7: Kết quả kiểm thử chống gian lận – đầy đủ 19 nhóm

Nhóm tấn công	Thành phần	Số test	Bị từ chối
reward	6	6	6/6
next_state	5	5	5/5
done flag	6	6	6/6
action	6	6	6/6
merkle_path	6	17	17/17
merkle_leaf	1	4	4/4
minibatch_index	8	23	23/23
q_value	2	5	5/5
td_target	2	4	4/4
loss	1	1	1/1
gradient	5	7	7/7
checkpoint_hash	8	15	15/15
target_network_sync	6	6	6/6
proof_public_input	12	44	42/42*
dataset_root	6	9	9/9
manifest_hash	8	8	8/8
raw_trajectory_hash	1	2	2/2
audit_report_hash	1	1	1/1
collection_log_hash	2	2	1/1**
Tổng cộng	—	166	166/166

* 2 hàng được phân loại không áp dụng (không có proof artifact).

** 1 hàng không áp dụng (dữ liệu công khai không có collection log).

Trong 166 trường hợp kiểm thử, không có trường hợp nào bị chấp nhận sai. Ba hàng được phân loại “không áp dụng” (2 ở nhóm proof_public_input, 1 ở nhóm collection_log_hash) không phải lỗi – chúng tương ứng với các cấu hình chưa có proof artifact hoặc không có collection log.

4.7 Artifact reproducibility

Sau khi hoàn thiện benchmark và proof provenance, dự án được đóng gói thêm artifact reproducibility layer. Mục tiêu của lớp này là cho phép người đọc hoặc người chấm chạy lại đường kiểm tra nhanh mà không cần sinh lại toàn bộ proof nặng. Đường chạy này đặc biệt quan trọng vì proof binaries không được commit vào repository, còn raw datasets và temporary work directories cũng được giữ ngoài version control.

Reviewer fast path là:

```
make reproduce-small
```

Lệnh này tạo hoặc kiểm tra tiny dataset audit report, validate compact SP1 provenance, kiểm tra report sources, sinh lại result summaries, kiểm tra theorem-artifact mapping và ghi artifact manifest. Với các tác vụ nặng hơn, dự án cung cấp các target riêng:

```
make reproduce-data-audit
make reproduce-sp1-proofs
make reproduce-benchmarks
make reproduce-tamper
make reproduce-paper-tables
make artifact-manifest
```

Các target nặng không được chạy mặc định. Chúng yêu cầu môi trường Linux, WSL2 hoặc Kaggle có SP1 toolchain và các biến môi trường tương ứng. Cách thiết kế này giúp dự án cân bằng giữa hai yêu cầu: một mặt giữ artifact đủ nhẹ để kiểm tra nhanh, mặt khác vẫn mô tả rõ đường chạy để sinh lại proof hoặc benchmark khi cần.

4.8 Tổng kết chương

Chương này đã trình bày cách dự án được triển khai và đánh giá. Mã nguồn được chia theo các lớp rõ ràng: relation, verifier, backend SP1, script dữ liệu, script thực nghiệm, test và final reports. Môi trường triển khai kết hợp Python, Rust, SP1, Kaggle CPU, Dockerfile và Makefile để hỗ trợ cả phát triển lẫn tái lập kết quả. Ba nhóm benchmark cho thấy hệ thống không chỉ có khả năng chạy các baseline RL, mà còn đo được chi phí proof và kiểm tra được khả năng reject các tamper đã định nghĩa. Phần artifact reproducibility giúp các kết quả này có thể được kiểm tra lại thông qua compact reports và các lệnh reproduce thay vì chỉ tồn tại dưới dạng mô tả trong báo cáo.

Chương 5

Đánh giá hệ thống, hạn chế và kết luận

5.1 Đánh giá tổng quan

Dự án xây dựng một pipeline kiểm chứng cho các hiện vật huấn luyện Offline DQN. Hệ thống không đề xuất thuật toán RL mới, mà kiểm tra các artifact tạo nên quá trình huấn luyện. Dữ liệu replay được ràng buộc bằng Merkle commitment; các phép tính TD và cập nhật trọng số được mô hình hóa thành relation; các relation trọng yếu được đưa sang SP1 zkVM để sinh và xác minh proof trong phạm vi đã giới hạn.

Ở lớp dữ liệu, hệ thống tách self-collected dataset khỏi public benchmark dataset. Với self-collected dataset, hệ thống có replay/reward audit trước khi commitment. Với public Minari/D4RL, hệ thống chỉ khẳng định source-integrity, không xem đó là honest collection proof. Merkle proof chỉ chứng minh tính nhất quán với dữ liệu đã cam kết; nó không chứng minh dữ liệu gốc được thu thập trung thực.

Ở lớp proof, một số relation chính đã có SP1 proof-backed evidence, bao gồm Merkle membership, TD MVP, Forward-TD MLP, one-step SGD tiny, short trace, one-step training update, training fragment với $k \in \{1, 4, 8\}$, và proof-manifest chunk-chain aggregation với $T \in \{32, 64, 128\}$. Một phần chỉ được gọi là proof-backed khi có public input, witness schema, metrics, verify report và tamper report tương ứng.

Ở lớp đánh giá, dự án có ba nhóm kết quả. Benchmark RL cho thấy các dataset đã cam kết vẫn có thể dùng để huấn luyện các baseline phù hợp. Benchmark proof cost ghi nhận verify time của các cấu hình proof-verified đều dưới 0.21 giây và proof size dưới 2.84 MB. Benchmark tamper rejection không ghi nhận trường hợp accepted unexpectedly trong các nhóm tấn công đã định nghĩa; các hàng not applicable được tách riêng khỏi lỗi chấp nhận sai.

Bảng 5.1: Tổng hợp các lớp kết quả chính của hệ thống

Lớp đánh giá	Kết quả đạt được	Ý nghĩa
Dữ liệu và commitment	Self-collected audit, public source-integrity, Merkle commitment	Ràng buộc dữ liệu replay với provenance trước khi proof.
Relation và proof	Nhiều relation có SP1 proof-backed evidence	Chuyển claim từ kiểm tra semantic sang proof-backed artifact.
Training fragment	$k \in \{1, 4, 8\}$ proof-backed	Kiểm tra chuỗi cập nhật ngắn thay vì một phép tính rời rạc.
Aggregation	$T \in \{32, 64, 128\}$ proof-manifest chain	Ràng buộc nhiều child manifest theo checkpoint link.
Tamper benchmark	Không có accepted unexpectedly; các hàng N/A được tách riêng	Kiểm tra hệ thống trước các sửa đổi có chủ đích.
Reproducibility	make reproduce-small	Cho phép kiểm tra nhanh compact provenance và report sources.

Dự án có đủ các thành phần của một security/systems artifact: pipeline, relation, backend, benchmark và artifact reproducibility. Phạm vi kết quả vẫn là selected Offline DQN training artifacts, không phải proof hoàn chỉnh cho mọi cấu hình huấn luyện DQN.

5.2 Công trình liên quan

Về Offline RL và DQN, DQN kết hợp value-function approximation, experience replay và target network cho temporal-difference learning. Double DQN tách next-action selection khỏi target-network value evaluation để giảm overestimation. Offline RL nghiên cứu bối cảnh học từ fixed dataset, nơi value learning dễ bị ảnh hưởng bởi distribution shift; CQL là một hướng giảm overestimation bằng value regularization. Dự án này không đề xuất thuật toán RL mới, mà làm cho một số phép tính DQN-style như replay membership, Bellman target và update evidence có thể kiểm chứng được trên dữ liệu đã cam kết.

Về proof-of-learning và proof-of-training, Proof-of-Learning đặt bài toán chứng minh mô hình đến từ một quá trình huấn luyện đã claim bằng cách ghi checkpoint trung gian và kiểm tra consistency. Tuy nhiên, hướng này không phải zero-knowledge và không cung cấp cryptographic soundness ở cùng mức với proof system. Các hệ thống như zkDL, Kaizen và

zkPoT mở rộng ZK proof cho huấn luyện deep learning, nhưng chủ yếu nhắm tới supervised learning hoặc mục tiêu khác; chúng không trực tiếp mã hóa các semantics đặc thù của Offline RL như terminal Bellman branch, replay membership, Double-DQN argmax, target-network synchronization và TD loss.

Về verifiable machine learning, nhiều công trình tập trung vào inference integrity như SafetyNets, zkCNN, VeriML và vCNN. Các hệ thống này chứng minh prediction hoặc out-sourced inference, trong khi relation chính của dự án là Bellman TD claim trên committed replay data. Một số hướng ZK cho federated learning kiểm tra gradient aggregation, có điểm tương đồng kỹ thuật với backpropagation và SGD update proof, nhưng threat model khác với bài toán malicious trainer trong Offline RL.

Về proof backend, general-purpose zkVM như SP1 và RISC Zero đánh đổi tối ưu circuit chuyên biệt để lấy tốc độ triển khai. Dự án dùng SP1 vì relation kết hợp hashing, Merkle path, schema check, integer arithmetic và branching. Aggregation hiện tại dùng proof-manifest chain, chưa dùng recursive proof verification. Nếu cần on-chain verification, một hướng tiếp theo là nén STARK proof sang Groth16 hoặc PLONK SNARK.

Bảng 5.2: So sánh định tính với các hệ thống kiểm chứng huấn luyện liên quan

Đặc trưng	PoL	zkDL/Kaizen/zkPoT	vCNN/zkCNN	Dự án này
ZK proof backend	–	Có	Có	Có
Training verification	Một phần	Có	–	Một phần
RL-specific relations	–	–	–	Có
Replay membership proof	–	–	–	Có
Bellman/TD target proof	–	–	–	Có
Target-network proof	–	–	–	Có
Dataset commitment	–	–	–	Có
Checkpoint chaining	Một phần	Có	–	Có
Multi-step composition	–	Có	–	Có
Recursive aggregation	–	Có ở một số hệ thống	–	Một phần
Tamper-rejection benchmark	–	–	–	Có
Artifact reproducibility	Một phần	Không rõ	Không rõ	Có

Điểm khác biệt chính của dự án là đặt Offline RL semantics làm first-class verification target. Các ký hiệu “Một phần” trong bảng trên cần được hiểu đúng phạm vi: với dự án này, training verification chỉ bao phủ selected fragments và proof-manifest aggregation, không phải end-to-end DQN training proof hay recursive child-proof verification.

5.3 Phân tích ưu điểm của hệ thống

Ưu điểm đầu tiên của dự án là kiến trúc phân lớp. Dataset provenance, relation semantics, proof backend, training fragment, aggregation và report generation được tách thành các phần tương đối độc lập. Khi một relation chưa có SP1 proof-backed evidence, hệ thống giữ nó ở mức Python semantic oracle hoặc execute-only thay vì đưa vào claim chính.

Ưu điểm thứ hai là khả năng truy vết claim. Một theorem không chỉ tồn tại dưới dạng

phát biểu hình thức, mà được map tới relation code, backend implementation, unit test, negative test và benchmark row. Người đọc có thể lần từ phát biểu lý thuyết tới file mã nguồn, artifact report và kết quả benchmark tương ứng.

Ưu điểm thứ ba là benchmark tamper được thiết kế sát với threat model. Các nhóm tamper không chỉ sửa dữ liệu đầu vào như reward, action, next state hay done flag, mà còn can thiệp vào Merkle path, TD target, Q value, gradient, checkpoint hash, target-network sync và proof public input. Điều này phản ánh đúng bản chất của Offline DQN: điểm gian lận có thể xuất hiện ở nhiều tầng, từ replay buffer cho tới checkpoint chain.

Ưu điểm cuối cùng là artifact reproducibility được đưa vào như một phần của hệ thống. Các lệnh reproduce nhỏ, lệnh sinh lại bảng kết quả và artifact manifest cho phép kiểm tra đường chạy nhanh mà không cần sinh lại toàn bộ proof nặng.

5.4 Hạn chế của hệ thống

Hạn chế lớn nhất của dự án là hệ thống chưa chứng minh toàn bộ quá trình huấn luyện DQN từ initialization đến final checkpoint. Các proof hiện tại chỉ bao phủ những relation và cấu hình đã có artifact cụ thể. Training fragment proof-backed mới dừng ở $k \in \{1, 4, 8\}$; các fragment dài hơn như $k \in \{16, 32, 128\}$ mới có reference hoặc execute-mode evidence, chưa có proof generation và verification provenance hoàn chỉnh.

Hạn chế thứ hai nằm ở optimizer và batch size. Training update hiện tập trung vào fixed-point SGD trên tiny Linear-ReLU-Linear MLP và batch-size 1. Trong các hệ thống DQN thực tế, Adam optimizer và batch size lớn thường được sử dụng để ổn định huấn luyện. Việc chứng minh Adam đòi hỏi kiểm tra first moment, second moment, bias correction và update rule cho từng tham số, nên phức tạp hơn đáng kể so với SGD.

Hạn chế thứ ba là aggregation hiện tại chưa phải true recursive aggregation. Proof-manifest chunk-chain aggregation ràng buộc child manifest hash, public-input hash, tính nhất quán dataset/config và checkpoint link, nhưng aggregate SP1 guest không verify trực tiếp child proof bytes. Vì vậy, claim đúng của hệ thống là proof-manifest chain, không phải recursive proof composition. Đây là giới hạn quan trọng, vì true recursion mới là hướng giúp verifier không phải kiểm nhiều child proof riêng lẻ.

Hạn chế thứ tư là benchmark RL và network architecture vẫn ở quy mô nhỏ. CartPole, MountainCar và PointMaze phù hợp cho artifact và kiểm tra pipeline, nhưng chưa đủ để kết luận hệ thống có thể scale trực tiếp lên Atari hoặc MuJoCo-scale training. Tương tự, tiny MLP giúp relation có thể proof được, nhưng chưa đại diện cho các kiến trúc lớn hơn như convolutional network trong Atari DQN.

Bảng 5.3: Giới hạn hiện tại và hướng khắc phục

Giới hạn	Tác động	Hướng xử lý
Chưa có full training proof	Chưa chứng minh toàn bộ trace từ W_0 đến final checkpoint	Mở rộng training fragment, thêm recursion và checkpoint coverage dài hơn.
Chưa có Adam proof	Chưa phản ánh đầy đủ cấu hình DQN thực tế	Mô hình hóa moment, bias correction và adaptive update bằng fixed-point.
Batch size còn nhỏ	Training update chưa giống setup huấn luyện thực tế	Thêm gradient accumulation cho batch size 4, 8, 16.
Chưa có true recursive aggregation	Aggregate proof chưa verify child proof bytes trong guest	Tích hợp recursive verifier hoặc SNARK-compressed child proofs.
Network còn tiny	Chưa áp dụng cho Atari-scale DQN	Tối ưu relation cho layer lớn hơn và tận dụng precompile/GPU proving.
Public dataset không có honest collection proof	Chỉ claim source-integrity với Minari/D4RL	Kết hợp cryptographic logging, TEE hoặc VRF cho self-collected data.

Các hạn chế trên xác định ranh giới claim của hệ thống. Với một hệ thống verifiable training, claim quá rộng sẽ dễ dẫn tới kết luận thiếu chính xác. Phần đánh giá vì vậy chỉ dựa trên các relation, backend và artifact đã triển khai.

5.5 Bài học rút ra

Quá trình thực hiện dự án giúp em rút ra ba bài học chính. Thứ nhất, kiểm chứng huấn luyện Offline RL không đơn giản chỉ là kiểm tra một công thức loss. Một bước cập nhật phụ thuộc vào rất nhiều yếu tố: tính hợp lệ của dữ liệu, tính toán Bellman, đánh giá mạng target, cho đến việc lan truyền ngược và lưu checkpoint. Chỉ cần bỏ sót một khâu, người chứng minh có thể lách luật. Vì thế, hệ thống cần thiết kế thành nhiều lớp chặt chẽ.

Thứ hai, việc sử dụng số học dấu phẩy tĩnh (fixed-point) là điều kiện bắt buộc để chạy proof. Trong các bài toán học máy thông thường, ta hay bỏ qua sai số dấu phẩy động. Nhưng ở hệ thống cryptographic, hai bên bắt buộc phải cho ra kết quả trùng khớp đến từng bit. Đổi lại, các khẳng định của dự án chỉ có giá trị trên các phép toán fixed-point đã cài đặt chứ không áp dụng chung cho mọi code PyTorch.

Cuối cùng, việc kiểm soát kỳ vọng và ranh giới đề tài rất quan trọng. Nếu khẳng định đây là "bằng chứng cho toàn bộ quá trình huấn luyện DQN", dự án sẽ bị đánh giá là chưa đạt

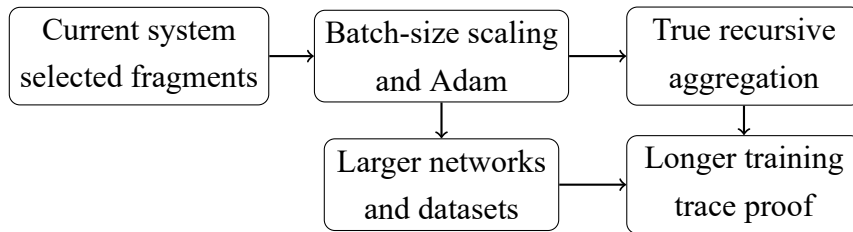
do thiếu optimizer phức tạp hay batch-size lớn. Nhưng khi thu hẹp lại thành ”kiểm chứng một số đoạn huấn luyện được chọn”, mọi tính năng em làm đều có minh chứng rõ ràng và sát thực tế.

5.6 Hướng phát triển

Từ những giới hạn hiện tại, em nhận thấy một số hướng mở rộng tiềm năng. Quan trọng nhất là hiện thực hóa tính năng tổng hợp đệ quy (true recursive aggregation). Hiện tại hệ thống mới chỉ liên kết các đoạn proof thông qua mã băm và file cấu hình bên ngoài. Nếu có thể xác minh các child proof trực tiếp ngay bên trong guest của zkVM, verifier sẽ chỉ cần kiểm tra một proof duy nhất cho toàn bộ quá trình dài.

Kế đến là việc mở rộng sang các thuật toán tối ưu thực tế hơn, tiêu biểu như Adam. Việc này sẽ phức tạp hơn SGD rất nhiều vì hệ thống phải lưu trữ, cập nhật trạng thái moment và xử lý các phép khai căn, chia tỷ lệ trong môi trường số nguyên fixed-point.

Ngoài ra, hệ thống cũng cần nâng cấp để hỗ trợ batch-size lớn hơn 1 (cộng dồn gradient) và kiến trúc mạng thần kinh phức tạp hơn. Để làm được điều đó, cần phải tối ưu mã Rust, giảm cycle count và có thể dùng đến GPU để sinh proof. Cuối cùng, khâu thu thập dữ liệu tự động (self-collected) có thể kết hợp thêm môi trường thực thi tin cậy (TEE) hoặc thuật toán chữ ký số để chứng minh quá trình tương tác môi trường ban đầu hoàn toàn minh bạch.



Hình 5.1: Các hướng phát triển từ artifact hiện tại tới proof cho trace dài hơn

5.7 Kết luận

Báo cáo này ghi lại toàn bộ quá trình thiết kế và xây dựng một hệ thống kiểm chứng đa lớp cho mô hình Offline DQN. Thay vì coi neural network là một ”hộp đen” và chỉ kiểm tra kết quả đầu ra, dự án đã chia nhỏ các bước huấn luyện thành những quan hệ toán học cụ thể và dùng bằng chứng không tiết lộ (ZKP) để xác minh tính chính xác của chúng.

Dù hiện tại hệ thống vẫn đang dừng ở dạng nguyên mẫu (prototype) với các giới hạn về batch size, thuật toán tối ưu và khả năng tổng hợp đệ quy, nhưng nó đã cung cấp một quy trình hoàn chỉnh từ lúc xử lý dữ liệu cho đến khi xuất báo cáo. Quá trình làm dự án không chỉ giúp em củng cố kiến thức về học máy mà còn mở ra cơ hội tiếp cận với kỹ thuật mật mã ứng dụng.

Tài liệu tham khảo

- [1] S. Levine, A. Kumar, G. Tucker, and J. Fu, “Offline reinforcement learning: Tutorial, review, and perspectives on open problems,” arXiv preprint arXiv:2005.01643, 2020.
- [2] H. Jia, M. Yaghini, C. A. Choquette-Choo, N. Dullerud, A. Thudi, V. Chandrasekaran, and N. Papernot, “Proof-of-learning: Definitions and practice,” in *IEEE Symposium on Security and Privacy*, 2021, pp. 1039–1056.
- [3] J. Thaler, *Proofs, Arguments, and Zero-Knowledge*. Foundations and Trends in Privacy and Security, 2022.
- [4] H. Sun, T. Bai, J. Li, and H. Zhang, “zkDL: Efficient zero-knowledge proofs of deep learning training,” Cryptology ePrint Archive, Paper 2023/1174, 2023.
- [5] K. Abbaszadeh, J. Katz, C. Pappas, and D. Papadopoulos, “Zero-knowledge proofs of training for deep neural networks,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 4316–4330.
- [6] G. Tan, A. Gascón, S. Meiklejohn, M. Raykova, X. Wang, and N. Luo, “Founding zero-knowledge proofs of training on optimum vicinity,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 1173–1187.
- [7] T. Liu, X. Xie, and Y. Zhang, “zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2968–2985.
- [8] Z. Ghodsi, T. Gu, and S. Garg, “SafetyNets: Verifiable execution of deep neural networks on an untrusted cloud,” in *Advances in Neural Information Processing Systems*, 2017, pp. 4675–4684.
- [9] S. Lee, H. Ko, J. Kim, and H. Oh, “vCNN: Verifiable convolutional neural network based on zk-SNARKs,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 5, pp. 4544–4558, 2024.
- [10] Succinct Labs, “Introducing SP1: A performant, 100% open-source, contributor-friendly zkVM,” 2024. [Online]. Available: <https://blog.succinct.xyz/introducing-sp1/>
- [11] RISC Zero, Inc., “RISC Zero zkVM: General purpose zero-knowledge computing,” 2023. [Online]. Available: <https://www.risczero.com/>

- [12] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [13] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.
- [14] P. J. Huber, “Robust estimation of a location parameter,” *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.
- [15] S. Fujimoto, D. Meger, and D. Precup, “Off-policy deep reinforcement learning without exploration,” in *Proceedings of the International Conference on Machine Learning*, 2019.
- [16] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative Q-learning for offline reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2020.
- [17] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Advances in Cryptology – EUROCRYPT 2016*. Springer, 2016, pp. 305–326.
- [18] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, “Scalable zero knowledge via cycles of elliptic curves,” in *Advances in Cryptology – CRYPTO 2014*. Springer, 2014, pp. 276–294.
- [19] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology – CRYPTO ’87*. Springer, 1987, pp. 369–378.
- [20] O. G. Younis, R. d. L. Balis, Y. Lu et al., “Minari: A standard library for offline reinforcement learning datasets,” in *NeurIPS Datasets and Benchmarks Track*, 2023.
- [21] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, “D4RL: Datasets for deep data-driven reinforcement learning,” arXiv preprint arXiv:2004.06120, 2020.
- [22] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, “OpenAI Gym,” arXiv preprint arXiv:1606.01540, 2016.
- [23] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.
- [24] L. Zhao, Q. Wang, C. Wang, Q. Li, C. Shen, X. Lin, S. Hu, and M. Du, “VeriML: Enabling integrity assurances and fair payments for machine learning as a service,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 10, pp. 2524–2540, 2021.

- [25] Z. Peng, C. Zhao, T. Wang, G. Liao, Z. Lin, Y. Liu, B. Cao, L. Shi, Q. Yang, and S. Zhang, “A survey of zero-knowledge proof based verifiable machine learning,” *Artificial Intelligence Review*, 2026.
- [26] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge,” IACR Cryptology ePrint Archive, Report 2019/953, 2019.
- [27] A. Verma, “Verifiable and interpretable reinforcement learning through program synthesis,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 9902–9903.
- [28] S. Longpre, R. Mahari, N. O.-M. Lee et al., “Position: Data authenticity, consent, and provenance for AI are all broken,” in *International Conference on Machine Learning*, 2024.
- [29] B. Alon, R. Dowsley et al., “A framework for cryptographic verifiability of end-to-end AI pipelines,” arXiv preprint arXiv:2503.18514, 2025.
- [30] M. Walfish and A. J. Blumberg, “Verifying computations without reexecuting them,” *Communications of the ACM*, vol. 58, no. 2, pp. 74–84, 2015.
- [31] Nguyễn Ngọc Duy, “Layered Zero-Knowledge Verifiable Artifacts for Offline Deep Q-Network Training over Committed Replay Data,” project artifact, 2026. [Online]. Available: https://github.com/pateel1811/zk_offline_dqn

Chương A

Các thuật toán kiểm chứng

Phụ lục này trình bày mã giả của ba thuật toán chính trong hệ thống kiểm chứng, bao gồm: Cam kết dữ liệu (Algorithm 1), Quy trình sinh và xác minh bằng chứng với SP1 (Algorithm 2), và Kiểm chứng một đoạn huấn luyện (Algorithm 3).

Algorithm 1 Cam kết dữ liệu replay có ràng buộc (Provenance-Bound Dataset Commitment)

Require: Dataset $\mathcal{D} = \{\tau_1, \dots, \tau_N\}$, manifest M , audit report A , collection log C (tùy chọn)

Ensure: Commitment $\mathcal{C}_D$

```
1: for  $i = 1$  to  $N$  do
2:    $\ell_i \leftarrow \text{SHA256}(\text{canonical\_encode}(\tau_i))$  ▷ Mã băm leaf của transition
3: end for
4:  $h_{\text{root}} \leftarrow \text{MerkleBuild}(\ell_1, \dots, \ell_N)$ 
5:  $h_M \leftarrow \text{SHA256}(\text{normalize}(M))$ 
6:  $h_A \leftarrow \text{SHA256}(A)$ 
7:  $h_R \leftarrow \text{SHA256}(\text{raw\_episodes})$ 
8:  $h_C \leftarrow \text{SHA256}(C)$  nếu  $C$  tồn tại, ngược lại  $\perp$ 
9: return  $\mathcal{C}_D = (h_{\text{root}}, h_M, h_A, h_R, h_C)$ 
```

Algorithm 2 Quy trình sinh/xác minh proof với SP1 (SP1 Relation Prove/Verify Workflow)

Require: JSON fixture F , $\text{mode} \in \{\text{execute}, \text{prove}\}$

Ensure: Phán quyết $\in \{\text{accept}, \text{reject}\}$, metrics M

- 1: Phân tích F thành public inputs x và witness w
 - 2: Biên dịch guest ELF binary
 - 3: **if** $\text{mode} = \text{execute}$ **then**
 - 4: Chạy guest natively: $v \leftarrow \text{Guest}(x, w)$
 - 5: **assert** $v.\text{ok} = \text{true}$
 - 6: **else if** $\text{mode} = \text{prove}$ **then**
 - 7: $(\pi, pv) \leftarrow \text{SP1.Prove}(\text{ELF}, x, w)$
 - 8: **assert** $\text{SP1.Verify}(\pi, pv)$
 - 9: **assert** $pv.\text{roots} = x.\text{roots}$ ▷ Ràng buộc public-input
 - 10: Ghi nhận M : cycle count, prove time, verify time, proof size
 - 11: **end if**
 - 12: **return** (accept, M)
-

Algorithm 3 Kiểm chứng đoạn huấn luyện k bước (k -Step Training Fragment Verification)

Require: Checkpoint đầu W_0 , target checkpoint $\bar{W}_0$, seed σ , Merkle root h_{root} , số bước k , hệ số scale S

Ensure: Checkpoint cuối W_k hợp lệ

```
1: for  $t = 0$  to  $k - 1$  do
2:    $i_t \leftarrow \text{DeterministicSample}(\sigma, t)$  ▷ Minibatch index
3:   assert  $R_{\text{mem}}(h_{\text{root}}; \tau_{i_t}, i_t, \pi_{i_t})$  ▷ Replay membership
4:    $Q_s \leftarrow \text{ForwardMLP}(W_t, s_{i_t})$  ▷ Q online tại state hiện tại
5:    $Q_{s'} \leftarrow \text{ForwardMLP}(W_t, s'_{i_t})$  ▷ Q online tại state tiếp theo
6:    $\bar{Q}_{s'} \leftarrow \text{ForwardMLP}(\bar{W}_t, s'_{i_t})$  ▷ Q target tại state tiếp theo
7:    $a^* \leftarrow \arg \max_{a'} Q_{s'}[a']$ 
8:    $y \leftarrow r_{i_t} + \gamma(1 - d_{i_t})\bar{Q}_{s'}[a^*]/S$ 
9:    $\ell \leftarrow L_{\text{SmoothL1}}(Q_s[a_{i_t}] - y)$ 
10:   $\nabla \leftarrow \text{Backprop}(W_t, \ell)$ 
11:   $W_{t+1} \leftarrow W_t - \eta \cdot \nabla / S$  ▷ Cập nhật SGD
12:  assert  $\text{SHA256}(W_{t+1}) = \text{claimed\_ckpt}_{t+1}$ 
13:  if  $(t + 1) \bmod T_{\text{sync}} = 0$  then
14:     $\bar{W}_{t+1} \leftarrow W_{t+1}$  ▷ Đồng bộ target network
15:  end if
16: end for
17: return  $W_k$ 
```
